

Nazwa kwalifikacji: **Projektowanie, programowanie i testowanie aplikacji**
Oznaczenie kwalifikacji: **INF.04**
Numer zadania: **03**

Wypełnia zdający

Numer Identyfikacyjny zdającego*

--	--	--	--	--	--	--	--

Miejsce na numer ośrodka

Czas trwania egzaminu: **180** minut.

INF.04-KG-26.03

EGZAMIN ZAWODOWY

Rok 2026

CZĘŚĆ PRAKTYCZNA

**PODSTAWA PROGRAMOWA
2019**

Instrukcja dla zdającego

1. Na pierwszej stronie arkusza egzaminacyjnego wpisz w oznaczonym miejscu swój numer Identyfikacyjny.
2. Na KARCIE OCENY wpisz swoje dane:
 - swój numer Identyfikacyjny,
 - oznaczenie kwalifikacji,
 - numer zadania,
 - numer stanowiska.
3. Sprawdź, czy arkusz egzaminacyjny zawiera 7 stron i nie zawiera błędów. Ewentualny brak stron lub inne usterki zgłoś przez podniesienie ręki przewodniczącemu zespołu nadzorującego.
4. Zapoznaj się z treścią zadania oraz stanowiskiem egzaminacyjnym. Masz na to 10 minut. Czas ten nie jest wliczany do czasu trwania egzaminu.
5. Czas rozpoczęcia i zakończenia pracy zapisze w widocznym miejscu przewodniczący zespołu nadzorującego.
6. Wykonaj samodzielnie zadanie egzaminacyjne. Przestrzegaj zasad bezpieczeństwa i organizacji pracy.
7. Po zakończeniu wykonania zadania pozostaw arkusz egzaminacyjny z rezultatami oraz KARTĘ OCENY na swoim stanowisku lub w miejscu wskazanym przez przewodniczącego zespołu nadzorującego.
8. Po uzyskaniu zgody zespołu nadzorującego możesz opuścić salę/miejsce przeprowadzania egzaminu.

Powodzenia!

* w przypadku braku numeru ID – seria i numer paszportu lub innego dokumentu potwierdzającego tożsamość

Zadanie egzaminacyjne

UWAGA: katalog z rezultatami pracy oraz płytę należy opisać numerem zdającego, którym został podpisany arkusz. Dalej w zadaniu numer ten jest nazwany numerem zdającego.

Wykonaj aplikację konsolową oraz webową według wskazań. Udokumentuj obie aplikacje zrzutami ekranu. Wykorzystaj konto **Egzamin** bez hasła.

Utwórz folder i nazwij go numerem zdającego. W folderze utwórz podfoldery: *konsola*, *webowa*, *testy*. Po wykonaniu każdej aplikacji, jej pełny kod (cały folder projektu) spakuj do archiwum. Następnie pozostaw w podfolderze jedynie pliki źródłowe, których treść była modyfikowana, plik wykonywalny, jeśli jest to możliwe oraz spakowane archiwum.

Część I. Aplikacja konsolowa

Za pomocą narzędzi do tworzenia aplikacji konsolowych zaimplementuj program służący do znajdowania daty świąt wielkanocnych.

Założenia programu

- Program wykonywany w konsoli.
- Język programowania zgodny z zainstalowanym na stanowisku egzaminacyjnym, jeden z: C++, C#, Java, Python.
- Podejście obiektowe.
- Program zawiera sprawdzanie dnia i miesiąca Wielkanocy.
- Program powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania kodu, należy stosować znaczące nazwy zmiennych i funkcji.

Metoda Meeusa/Jonesa/Butchera

1. Dzielimy liczbę roku na 19 i wyznaczamy resztę a.
2. Dzielimy liczbę roku przez 100, wynik zaokrąglamy w dół (odcinamy część ułamkową) i otrzymujemy liczbę b.
3. Dzielimy liczbę roku przez 100 i otrzymujemy resztę c.
4. Liczymy: $b: 4$ i wynik zaokrąglamy w dół i otrzymujemy liczbę d.
5. Liczymy: $b: 4$ i wyznaczamy resztę e.
6. Liczymy: $(b + 8): 25$ i wynik zaokrąglamy w dół i otrzymujemy liczbę f.
7. Liczymy: $(b - f + 1): 3$ i wynik zaokrąglamy w dół i otrzymujemy liczbę g.
8. Liczymy: $(19 \times a + b - d - g + 15): 30$ i wyznaczamy resztę h.
9. Liczymy: $c: 4$ i wynik zaokrąglamy w dół i otrzymujemy cyfrę i.
10. Liczymy: $c: 4$ i wyznaczamy resztę k.
11. Liczymy: $(32 + 2 \times e + 2 \times i - h - k): 7$ i otrzymujemy resztę l.
12. Liczymy: $(a + 11 \times h + 22 \times l): 451$ i wynik zaokrąglamy w dół i otrzymujemy liczbę m.
13. Liczymy: $(h + l - 7 \times m + 114): 31$ i otrzymujemy resztę p.
14. Dzień Wielkanocy = $p + 1$.
15. Miesiąc = Zaokrąglenie w dół dzielenia $(h + l - 7 \times m + 114)$ przez 31.

Źródło: <https://pl.wikipedia.org/wiki/Wielkanoc> ; dostęp: 25.01.2026

Instrukcja krok po kroku 1 - Metoda Meeusa/Jonesa/Butchera

Opis programu

Program składa się z Klasy która posiada metodę sprawdzania dnia Wielkanocy metodą Meeusa/Jonesa/Butchera.

Założenia dotyczące kodu

- Program zawiera klasę Wielkanoc która posiada metodę obliczającą dzień Wielkanocy.
- Metoda obliczająca posiada:
 - argument liczbę całkowitą - rok
 - implementację algorytmu Meeusa/Jonesa/Butchera (zgodnie z instrukcją krokową powyżej).
 - zwraca tekst "<dzien> <miesiac>" gdzie <dzien> to liczba, a <miesiac> to nazwa miesiąca (tutaj wykorzystujemy tylko: „marca”, „kwietnia”)
- Nie jest wymagana walidacja danych – zakłada się, że wprowadzono go bez błędów

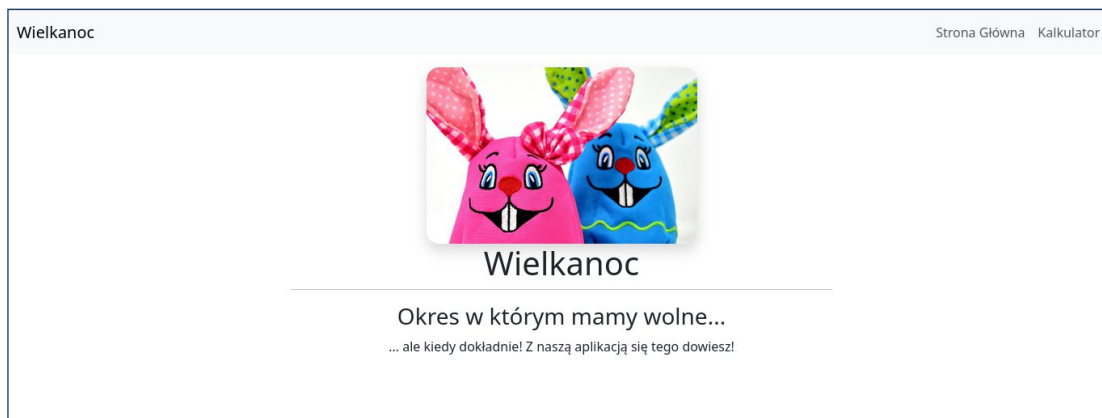
Sprawdź działanie metody lub funkcji sprawdzającej. Program powinien nawiązać czytelną komunikację z użytkownikiem.

Po uruchomieniu programu wykonaj zrzuty ekranu dokumentujące interakcje programu dla różnych wartości roku. Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym. Liczba zrzutów ekranu powinna odpowiadać wszystkim możliwym interakcjom użytkownika z programem. Zrzuty ekranu zapisz w folderze konsolowa pod nazwami *konsola1.png*, *konsola2.png*, itd.

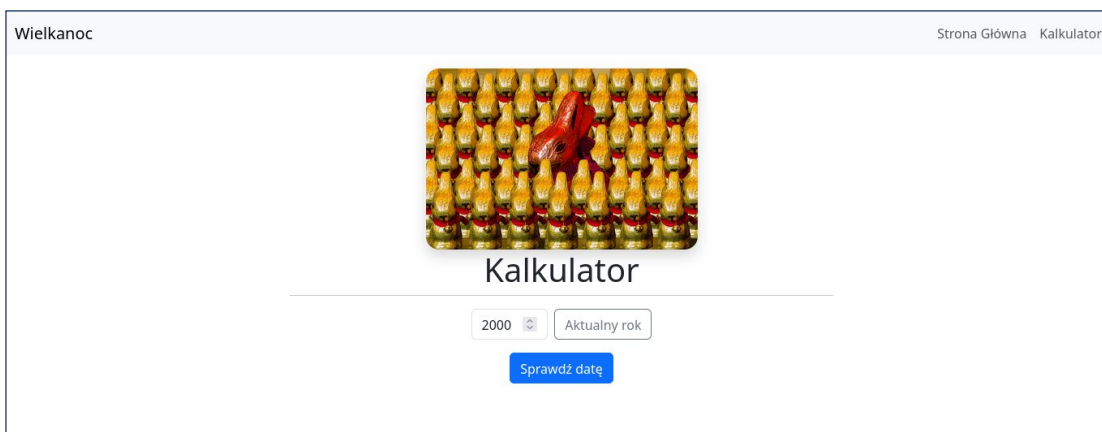
Kod aplikacji przygotuj do nagrania na płytę. W folderze konsolowa powinno znaleźć się archiwum całego projektu o nazwie *konsola.zip*, skopiowany z projektu plik z kodem źródłowym programu oraz plik wykonywalny, jeżeli istnieje.

Część II. Aplikacja webowa

Za pomocą środowiska programistycznego dostępnego na stanowisku egzaminacyjnym, wykonaj aplikację webową będącą graficznym interfejsem do sprawdzania dnia Wielkanocy. Aplikacja wykorzystuje kod utworzony w części I (aplikacja konsolowa). Do zbudowania aplikacji wykorzystaj grafiki z archiwum **pliki3.7z** zabezpieczonego hasłem: **W13lkan0c&**



Obraz 1. Aplikacja webowa, w stanie początkowym



Obraz 2. Aplikacja webowa, na stronie kalkulatora

Na obrazie 1 i 2 przedstawiono ideę aplikacji. W zależności od zastosowanego środowiska programistycznego wygląd może nieznacznie się różnić.

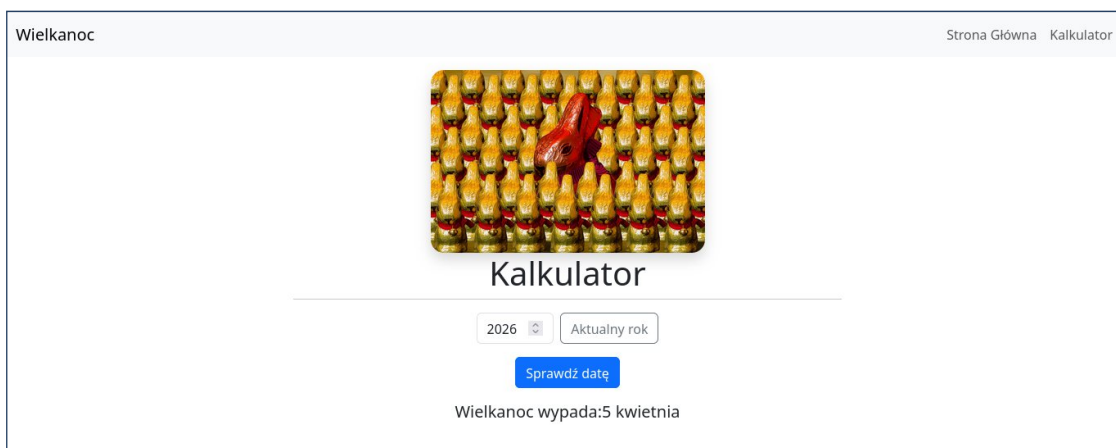
Opis wyglądu aplikacji

- Aplikacja składa się z widoków (tras), głównej (/) i kalkulatora (/calculator) które są obsługiwane przy pomocy nawigacji znajdującej się w nagłówku strony
- Na widoku głównym znajduje się:
 - obrazu *easter.jpg*, zaokrąglonego z cieniem
 - tekstu *Wielkanoc*, większego od pozostałych
 - kreski poziomej
 - tekstu: *Okres w którym mamy wolne...*
 - tekstu: *... Ale kiedy dokładnie? Z naszą aplikacją się tego dowiesz!*
- Widok kalkulatora składa się z:
 - obrazu *easter2.jpg*, zaokrąglonego z cieniem
 - tekstu *Kalkulator*, większego od pozostałych
 - kreski poziomej
 - pola wprowadzania typu numerycznego z granicami 2000 - 3000
 - przycisku zmiany roku na aktualny, obramowanego

- przycisku obliczania wyniku
- pustego pola w którym zostanie pokazany wynik
- Komponent *Header* który ma szare tło skład się z:
 - tekstu po lewej stronie o treści: *Wielkanoc*,
 - menu po prawej stronie z odnośnikami do tras: *Strona Główna*, *Kalkulator*

Opis działania aplikacji

- Po uruchomieniu aplikacji domyślnie znajduje się na stronie głównej
- Na widoku kalkulatora po naciśnięciu przycisku *Aktualny rok* zmienia się data w polu numerycznym na aktualny rok
- Przycisk *Sprawdź datę* pobiera podany rok z pola numerycznego i wykonuje skrypt obliczający datę Wielkanocy (zgodnie z częścią I) (przykład wyniku został pokazany na obrazie 3).



Obraz 3. Aplikacja webowa, na stronie kalkulatora (wynik)

Założenia aplikacji

- Aplikacja składa się z dwóch widoków (tras) obsługiwanych przez router:
 - Strona główna dostępna pod ścieżką główną /
 - Kalkulator dostępny pod ścieżką /calculator
- Do aplikacji został użyty Bootstrap, bez użycia własnego CSS-a.
- Kod aplikacji powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania
- Należy stosować znaczące nazwy zmiennych, metod i kontrolek
- Grafiki wykorzystywane w aplikacji pochodzą z archiwum dołączonego do zadania egzaminacyjnego

Podejmij próbę kompilacji i uruchomienia aplikacji. Wykonaj zrzut ekranowy stanu początkowego aplikacji i stanu po sprawdzeniu dnia Wielkanocy. Zrzuty ekranu zapisz w folderze webowa pod nazwami: *webowa1.png*, *webowa2.png*

Kod aplikacji przygotuj do nagrania na płytę. W podfolderze webowa powinno znaleźć się archiwum całego projektu o nazwie *webowa.zip* oraz plik (lub pliki) z kodem źródłowym modyfikowanym w czasie egzaminu.

Część III. Testowanie aplikacji webowej

Wykonaj testy jednostkowe metody klasy Wielkanoc, utworzonej w części II zadania (aplikacja webowa). Do napisania testów wykorzystaj dostępną na stanowisku egzaminacyjnym bibliotekę do testów jednostkowych. Testy jednostkowe powinny sprawdzać poprawność obliczania dnia Wielkanocy.

Założenia testów

- W każdej metodzie testującej należy użyć asercji
- Każdy przypadek testowy jest realizowany w osobnej metodzie.
- Nazwy metod testujących powinny jasno określać cel testu
- Wykonuje testy z tabeli 1

Tabela 1. Scenariusz testowania z przypadkami testowymi

Nr testu	Cel testu (Jaka sytuacja będzie testowana)	Dane wejściowe	wartość
1	Sprawdzenie dnia Wielkanocy w roku 2026	2026	5 kwietnia (oczekiwana)
2	Sprawdzenie dnia Wielkanocy w roku 2026	2026	6 kwietnia (nie oczekiwana)
3	Sprawdzenie dnia Wielkanocy w roku 2000	2000	23 kwietnia (oczekiwana)

* test 2 sprawdza czy wynik nie jest równy

Uruchomienie testów i dokumentacja:

- Po napisaniu metod testujących uruchom wszystkie testy jednostkowe
- Wykonaj zrzut dokumentujący wyniki wykonania metod testujących
- Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym
- Zrzuty zapisz w folderze testy pod nazwami: *test1.png* itd.

Utwórz archiwum ZIP zawierające projekt testowy (jeśli to potrzebne) i zapisz w folderze testy. W folderze z numerem zdającego utwórz plik pakietu biurowego o nazwie **egzamin** i umieść w nim:

- nazwę systemu operacyjnego, na którym pracował zdający,
- nazwy środowisk programistycznych, z których zdający korzystał na egzaminie,
- nazwy języków programowania użytych podczas tworzenia aplikacji

Tabela 2. Selected React-Router-Dom & Vitest methods

Name	Description
<Link to={{pathname : "..."}} > </Link>	A React Router component used for navigation between pages in an application without reloading the page
<BrowserRouter>...</BrowserRouter>	The main React Router component that uses the HTML5 History API to synchronize the user interface with the browser's URL.
<Routes> ... </Routes>	A React Router component containing a collection of routes
<Route path="..." element={...} />	A React Router component that defines a single route in the application. Specifies the URL path and the component to be rendered when that path is matched.
test("...", callback);	A Vitest function that defines a single test case. Takes a test name as the first parameter and a callback function containing test assertions as the second parameter.
expect(value)	A Vitest assertion function that wraps a value to be tested. Returns an object with matcher methods for performing assertions on the wrapped value.
.toBe(value)	A Vitest matcher that checks if the expected value is strictly equal to the received value using the === operator.

UWAGA: Nagraj płytę z rezultatami pracy. W folderze z numerem zdającego powinny się znajdować podfoldery: konsola, webowa, testy i plik egzamin. W folderze konsola: cały projekt aplikacji konsolowej i zrzuty. W folderze webowa: cały projekt aplikacji webowej i zrzuty. W folderze testy: cały kod testów/archiwum aplikacji testującej, ewentualnie inne przygotowane pliki. Po nagraniu płyty sprawdź poprawność nagrania. Opisz płytę numerem zdającego i pozostaw na stanowisku, zapakowaną w pudełku wraz z arkuszem egzaminacyjnym.

Czas przeznaczony na wykonanie zadania wynosi 180 minut.

Ocenie będą podlegać 4 rezultaty:

- implementacja, kompilacja, uruchomienie programu,
- aplikacja konsolowa,
- aplikacja webowa,
- testy aplikacji